CÓDIGOCLARO · RUTA REACT

Tema 14 · Pruebas de componentes

Guía de estudio · DAM/DAW

Testing Library, interacción y accesibilidad.

1. Explicación

React Testing Library consulta el DOM con criterios parecidos a los de una persona, como el nombre accesible de un botón. Las pruebas deben centrarse en comportamiento observable y evitar acoplarse a variables internas o clases de implementación. user-event puede modelar acciones como clics y escritura; las pruebas asíncronas deben esperar el cambio. El runner y el entorno DOM, por ejemplo Vitest o Jest, se configuran en el proyecto.

2. Ejemplo

Código · jsx

```jsx
// Contador.jsx
import { useState } from "react";
export function Contador() {
  const [valor, setValor] = useState(0);
  return <button onClick={() => setValor(n => n + 1)}>Añadir: {valor}</button>;
}

// Contador.test.jsx
import { expect, test } from "vitest";
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { Contador } from "./Contador";
test("el botón incrementa el valor visible", async () => {
  const usuario = userEvent.setup();
  render(<Contador />);
  await usuario.click(screen.getByRole("button", { name: "Añadir: 0" }));
  expect(screen.getByRole("button", { name: "Añadir: 1" })).toBeTruthy();
});
```

Contexto: Requiere React Testing Library, user-event, Vitest y un entorno DOM de prueba configurado.

Resultado esperado

```
La prueba pasa después de que el nombre accesible del botón cambia de Añadir: 0 a Añadir: 1.
```

3. Errores frecuentes

- Buscar nodos por clases CSS internas si se puede consultar el nombre accesible.
- No esperar una interacción o resultado asíncrono.
- Probar que se llamó una función interna sin comprobar el resultado que ve la persona.

4. Ejercicio

Añade pruebas para un botón de alternancia cuyo texto visible cambia entre Mostrar y Ocultar.

Casos que debe superar

Entrada	Resultado esperado
Render inicial	Existe el botón Mostrar
Un clic	Existe el botón Ocultar
Dos clics	Vuelve a existir el botón Mostrar

5. Solución explicada

Código · jsx

```jsx
import { expect, test } from "vitest";
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";

test("al alternar, cambia el nombre accesible", async () => {
  const usuario = userEvent.setup();
  render(<Alternador />);
  await usuario.click(screen.getByRole("button", { name: "Mostrar" }));
  expect(screen.getByRole("button", { name: "Ocultar" })).toBeTruthy();
});
```

Criterio de aceptación: Se aceptan consultas equivalentes y pruebas separadas por caso si siguen el flujo que usaría una persona.

Otras soluciones válidas

- Consultar por rol y nombre accesible con user-event.
- Consultar por label para controles de formulario y usar fireEvent para eventos específicos.

6. Fuentes oficiales

Testing Library — React Testing Library · Testing Library — User Interactions